프로토콜 업그레이드 체인 문서화

개요

이 문서는 Rust libp2p에서 프로토콜 업그레이드 체인(Protocol Upgrade Chain)의 구조와 동작 방식을 상세히 분석합니다. 업그레이드 체인은 원시 네트워크 연결을 단계적으로 변환하여 최종적으로 애플리케이션 프로토콜이 사용할 수 있는 형태로 만드는 과정입니다.

업그레이드 체인 구조

1. 전체 업그레이드 체인 개요

```
Raw Transport Connection

↓

Version Negotiation (multistream-select)

↓

Security Upgrade (Noise/TLS)

↓

Multiplexing Upgrade (Yamux/Mplex)

↓

Application Protocol Negotiation

↓

Application Protocol Handler
```

2. 각 단계별 상세 분석

2.1 Raw Transport Connection

- **입력**: 네트워크 소켓 (TCP, QUIC, WebSocket 등)
- **출력**: AsyncRead + AsyncWrite 트레이트를 구현하는 스트림
- **역할**: 기본 네트워크 연결 제공

2.2 Version Negotiation

- **입력**: 원시 연결 스트림
- **출력**: 버전이 협상된 연결
- **프로토콜**: multistream-select
- **역할**: 업그레이드 프로토콜 버전 협상

2.3 Security Upgrade

- **입력**: 버전 협상된 연결
- **출력**: (PeerId, 암호화된 연결) 튜플
- **프로토콜**: Noise Protocol, TLS
- **역할**: 피어 인증 및 암호화 채널 설정

2.4 Multiplexing Upgrade

- **입력**: (PeerId, 암호화된 연결) 튜플
- **출력**: (PeerId, StreamMuxer) 튜플
- **프로토콜**: Yamux, Mplex
- **역할**: 단일 연결에서 다중 스트림 지원

2.5 Application Protocol Negotiation

- **입력**: 개별 서브스트림
- **출력**: 협상된 프로토콜 핸들러
- **프로토콜**: multistream-select (서브스트림 레벨)
- **역할**: 애플리케이션 프로토콜 선택

업그레이드 체인 구현 분석

1. Builder 패턴을 통한 타입 안전한 체인 구성

```
// libp2p-core/src/transport/upgrade.rs
pub struct Builder<T> {
inner: T,
version: upgrade::Version,
}

impl<T> Builder<T> {
pub fn authenticate<C, D, U, E>(self, upgrade: U) -> Authenticated<...> {
// 인증 업그레이드 적용
}
}

pub struct Authenticated<T>(Builder<T>);

impl<T> Authenticated<T> {
```

```
pub  fn multiplex<C, M, U, E>(self, upgrade: U) -> Multiplexed<...> {
// 멀티플렉싱 업그레이드 적용
}
}
```

특징:

- **타입 안전성**: 컴파일 타임에 업그레이드 순서 검증
- **단계별 제약**: 인증 → 멀티플렉싱 순서 강제
- **불변성**: 각 단계에서 새로운 타입 반환

2. 업그레이드 적용 메커니즘

2.1 upgrade::apply 함수

```
// libp2p-core/src/upgrade/apply.rs
pub(crate) fn apply<C, U>(
conn: C,
up: U,
cp: ConnectedPoint,
v: Version,
) -> Either<InboundUpgradeApply<C, U>, OutboundUpgradeApply<C, U>>
where
C: AsyncRead + AsyncWrite + Unpin,
U: InboundConnectionUpgrade<Negotiated<C>> + OutboundConnectionUpgrade<Negotiate
{
```

```
match cp {
ConnectedPoint::Dialer { role_override, .. } if role_override.is_dialer() => {
Either::Right(apply_outbound(conn, up, v))
}
_ => Either::Left(apply_inbound(conn, up)),
}
}
```

역할 기반 업그레이드:

- **Dialer (클라이언트)**: OutboundUpgrade 사용
- **Listener (서버)**: InboundUpgrade 사용
- **비대칭 처리**: 각 역할에 맞는 핸드셰이크 수행

2.2 InboundUpgradeApply 구조

```
pub  struct InboundUpgradeApply<C, U>
where
C: AsyncRead + AsyncWrite + Unpin,
U: InboundConnectionUpgrade<Negotiated<C>>,
{
inner: InboundUpgradeApplyState<C, U>,
}

enum InboundUpgradeApplyState<C, U> {
```

```
Init {
future: ListenerSelectFuture<C, U::InfoIter>,
upgrade: U,
},
Upgrade {
future: U::Future,
},
Invalid,
}
```

상태 기반 처리:

1. **Init**: 프로토콜 네고시에이션 단계
2. **Upgrade**: 실제 업그레이드 수행 단계
3. **Invalid**: 오류 상태

3. 프로토콜 네고시에이션 (multistream-select)

3.1 UpgradeInfo 트레이트

```rust
pub trait UpgradeInfo {
type Info: AsRef<str> + Clone;
type InfoIter: IntoIterator<Item = Self::Info>;

fn protocol_info(&self) -> Self::InfoIter;
```

```
}
```

프로토콜 식별:

- **프로토콜 문자열**: “/noise”, “/yamux/1.0.0” 등
- **버전 구분**: 동일 프로토콜의 다른 버전은 별개 프로토콜로 처리
- **다중 지원**: 하나의 업그레이드가 여러 프로토콜 지원 가능

3.2 네고시에이션 과정

```
Dialer Listener

| |

| "/multistream/1.0.0" |

|-------------------------------->|

| |

| "/noise" |

|-------------------------------->|

| |

| "/noise" |

|<--------------------------------|

| |

| (Noise handshake begins) |

|<================================>|
```

4. 보안 업그레이드 체인

4.1 Noise Protocol 업그레이드

```
impl<T> InboundConnectionUpgrade<T> for noise::Config

where

T: AsyncRead + AsyncWrite + Unpin + Send + 'static,

{

type Output = (PeerId, Output<T>);

type Error = Error;

type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S



fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {

async  move {

let  mut state = self.into_responder(socket)?;

// XX 핸드셰이크 패턴 실행

handshake::recv_empty(&mut state).await?; // 1단계

handshake::send_identity(&mut state).await?; // 2단계

handshake::recv_identity(&mut state).await?; // 3단계

let (pk, io) = state.finish()?;

Ok((pk.to_peer_id(), io))

}.boxed()

}

}
```

업그레이드 변환:

- **입력 타입**: T (원시 연결)
- **출력 타입**: (PeerId, Output) (인증된 연결)
- **부가 정보**: 피어 신원 확인 및 암호화 설정

4.2 TLS 업그레이드 (대안)

```
impl<T> InboundConnectionUpgrade<T> for tls::Config
where
T: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = (PeerId, tls::TlsStream<T>);
type Error = tls::Error;
type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S


fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {
async  move {
let tls_stream = self.accept(socket).await?;
let peer_id = extract_peer_id_from_certificate(&tls_stream)?;
Ok((peer_id, tls_stream))
}.boxed()
}
}
```

5. 멀티플렉싱 업그레이드 체인

5.1 Yamux 업그레이드

```
impl<C> InboundConnectionUpgrade<C> for yamux::Config
where
C: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = yamux::Muxer<C>;
type Error = io::Error;
type Future = future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_inbound(self, socket: C, _: Self::Info) -> Self::Future {
let config = self.into_yamux_config();
let connection = yamux::Connection::new(socket, config, yamux::Mode::Server);
future::ok(yamux::Muxer::new(connection))
}
}
```

업그레이드 변환:

- **입력 타입**: C (인증된 연결)
- **출력 타입**: yamux::Muxer (멀티플렉서)
- **역할 구분**: Server/Client 모드 설정

5.2 StreamMuxer 인터페이스

```rust
pub trait StreamMuxer {
type Substream;
type Error;

fn poll_inbound(&mut self, cx: &mut Context<'_>) -> Poll<Result<Self::Substream
fn poll_outbound(&mut self, cx: &mut Context<'_>) -> Poll<Result<Self::Substrea
fn poll_close(&mut self, cx: &mut Context<'_>) -> Poll<Result<(), Self::Error>>
}
```

업그레이드 체인 조합 패턴

1. 표준 TCP + Noise + Yamux 체인

```rust
use libp2p::{tcp, noise, yamux, Transport};

let transport = tcp::Transport::default()
.upgrade(upgrade::Version::V1)
.authenticate(noise::Config::new(&keypair)?)
.multiplex(yamux::Config::default())
.boxed();
```

체인 구성:

1. **TCP Transport**: 기본 TCP 연결

2. **Version V1**: multistream-select v1.0.0

3. **Noise Authentication**: XX 핸드셰이크 + 암호화

4. **Yamux Multiplexing**: 스트림 멀티플렉싱

5. **Boxing**: 타입 소거를 통한 동적 디스패치

2. QUIC 기반 체인 (내장 보안)

```
let transport = quic::Transport::new(quic::Config::new(&keypair))
.multiplex(yamux::Config::default()) // QUIC은 이미 보안 내장
.boxed();
```

특징:

- **내장 보안**: QUIC은 TLS 1.3 내장으로 별도 인증 업그레이드 불필요
- **내장 멀티플렉싱**: QUIC 자체가 멀티스트림 지원하지만 libp2p 호환성을 위해 Yamux 추가

3. WebSocket + TLS + Mplex 체인

```
let transport = websocket::WsConfig::new(tcp::Transport::default())
.upgrade(upgrade::Version::V1)
.authenticate(tls::Config::new(&keypair)?)
.multiplex(mplex::Config::default())
.boxed();
```

업그레이드 체인 실행 흐름

1. 연결 설정 시 업그레이드 실행

```
// Transport::dial 호출 시
async fn dial_with_upgrades(addr: Multiaddr) -> Result<(PeerId, StreamMuxerBox)
// 1. 기본 연결 설정
let raw_conn = tcp_transport.dial(addr).await?;
// 2. 보안 업그레이드 적용
let (peer_id, secure_conn) = noise_upgrade.upgrade_outbound(raw_conn, "/noise").
// 3. 멀티플렉싱 업그레이드 적용
let muxer = yamux_upgrade.upgrade_outbound(secure_conn, "/yamux/1.0.0").await?;
Ok((peer_id, muxer.boxed()))
}
```

2. 리스너에서의 업그레이드 처리

```
// Transport::listen_on 후 연결 수락 시
async fn accept_with_upgrades(raw_conn: TcpStream) -> Result<(PeerId, StreamMux
// 1. 보안 업그레이드 적용 (서버 모드)
let (peer_id, secure_conn) = noise_upgrade.upgrade_inbound(raw_conn, "/noise").a
// 2. 멀티플렉싱 업그레이드 적용 (서버 모드)
let muxer = yamux_upgrade.upgrade_inbound(secure_conn, "/yamux/1.0.0").await?;
Ok((peer_id, muxer.boxed()))
```

```
}
```

3. 서브스트림 레벨 프로토콜 네고시에이션

```
// 애플리케이션 프로토콜 사용 시
async  fn negotiate_application_protocol(
substream: impl AsyncRead + AsyncWrite + Unpin
) -> Result<KademliaStream, ProtocolError> {
// multistream-select를 통한 프로토콜 네고시에이션
let negotiated = multistream_select::dialer_select_proto(
substream,
["/kad/1.0.0"],
Version::V1
).await?;
// Kademlia 프로토콜 핸드셰이크
let kad_stream = kademlia::upgrade_outbound(negotiated.1, negotiated.0).await?;
Ok(kad_stream)
}
```

업그레이드 체인 오류 처리

1. 업그레이드 오류 타입

```
pub enum UpgradeError<E> {
Select(NegotiationError), // 프로토콜 네고시에이션 실패
Apply(E), // 업그레이드 적용 실패
}

pub enum NegotiationError {
Failed, // 공통 프로토콜 없음
ProtocolError(ProtocolError), // 프로토콜 형식 오류
}
```

2. 체인별 오류 처리 전략

```
match transport.dial(addr).await {
Ok((peer_id, muxer)) => {
// 성공적인 연결
println!("Connected to {peer_id}");
}
Err(TransportError::MultiaddrNotSupported(addr)) => {
// 지원하지 않는 주소 형식
eprintln!("Unsupported address: {addr}");
}
Err(TransportError::Other(upgrade_error)) => {
match upgrade_error.downcast_ref::<UpgradeError<noise::Error>>() {
```

```
Some(UpgradeError::Select(neg_err)) => {
eprintln!("Protocol negotiation failed: {neg_err}");
}
Some(UpgradeError::Apply(noise_err)) => {
eprintln!("Noise handshake failed: {noise_err}");
}
_ => {
eprintln!("Other upgrade error: {upgrade_error}");
}
}
}
}
}
```

커스텀 업그레이드 구현

1. 커스텀 보안 업그레이드 예시

```
pub  struct CustomAuth {
keypair: identity::Keypair,
}

impl UpgradeInfo for CustomAuth {
type Info = &'static str;
```

```
type InfoIter = std::iter::Once<Self::Info>;


fn protocol_info(&self) -> Self::InfoIter {
std::iter::once("/custom-auth/1.0.0")
}
}


impl<T> InboundConnectionUpgrade<T> for CustomAuth
where
T: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = (PeerId, AuthenticatedStream<T>);
type Error = AuthError;
type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S


fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {
async  move {
// 커스텀 인증 프로토콜 구현
let  mut stream = AuthenticatedStream::new(socket);
// 1. 챌린지 수신
let challenge = stream.read_challenge().await?;
// 2. 서명 생성 및 전송
let signature = self.keypair.sign(&challenge);
```

```
stream.send_signature(signature).await?;

// 3. 상대방 서명 검증

let peer_signature = stream.read_signature().await?;

let peer_id = verify_signature(challenge, peer_signature)?;

Ok((peer_id, stream))

}.boxed()

}

}
```

2. 업그레이드 체인에 통합

```
let transport = tcp::Transport::default()

.upgrade(upgrade::Version::V1)

.authenticate(CustomAuth::new(keypair)) // 커스텀 인증 사용

.multiplex(yamux::Config::default())

.boxed();
```

성능 최적화 고려사항

1. 업그레이드 체인 최적화

```
// 불필요한 중간 할당 방지

let transport = tcp::Transport::default()
```

```
.upgrade(upgrade::Version::V1)
.authenticate(noise::Config::new(&keypair)?)
.multiplex(yamux::Config::default())
.map(|(peer_id, muxer), _| (peer_id, StreamMuxerBox::new(muxer))) // 조기 박싱
.boxed();
```

2. 메모리 사용량 최적화

- **스트림 버퍼 크기**: Yamux/Mplex 설정에서 적절한 버퍼 크기 설정
- **연결 풀링**: 동일 피어에 대한 연결 재사용
- **타임아웃 설정**: 장시간 대기하는 업그레이드 방지

3. 네트워크 효율성

- **프로토콜 우선순위**: 선호하는 프로토콜을 먼저 네고시에이션
- **병렬 업그레이드**: 가능한 경우 동시 업그레이드 수행
- **캐싱**: 반복적인 네고시에이션 결과 캐싱

결론

Rust libp2p의 프로토콜 업그레이드 체인은 다음과 같은 핵심 특징을 가집니다:

주요 장점

1. **타입 안전성**: 컴파일 타임에 업그레이드 순서와 호환성 검증
2. **모듈성**: 각 업그레이드 단계를 독립적으로 구현 및 테스트 가능
3. **확장성**: 새로운 프로토콜과 업그레이드를 쉽게 추가

4. **견고성**: 포괄적인 오류 처리와 복구 메커니즘

5. **성능**: 제로 코스트 추상화와 효율적인 메모리 사용

설계 원칙

1. **계층적 구조**: 명확한 책임 분리와 단계별 변환

2. **비동기 처리**: 모든 업그레이드가 논블로킹 방식으로 동작

3. **역할 기반**: Dialer/Listener 역할에 따른 적절한 핸드셰이크

4. **프로토콜 중립성**: 다양한 프로토콜 조합 지원

5. **오류 투명성**: 각 단계의 오류를 명확히 구분하여 처리

이러한 설계를 통해 libp2p는 복잡한 P2P 네트워킹 요구사항을 만족하면서도 개발자에게 직관적이고 안전한 API를 제공합니다.